

Waveform Analysis Using Testbenches	2
1. Title	2
2. Objective	2
3. Theory (Multi-Level)	2
3.1 Beginner — Simple Intuition.....	2
3.2 Intermediate — Engineering View.....	2
3.3 Advanced — Signal/Architecture Level	3
4. Architecture / Concept.....	5
GTKWave Analysis Layers	5
Waveform Navigation Workflow.....	6
5. Implementation.....	6
Testbench Files for This Module.....	6
Pre-saved GTKWave Layout.....	7
6. Lab / Hands-on Section	7
Lab A: Full SoC Waveform Analysis.....	7
Lab B: AXI Handshake Analysis.....	8
Lab C: Protocol Debug Scenario.....	8
7. Waveform / Verification Insight.....	8
Checklist: What to Verify in tb_top.fst	8
Timing Reference Table	9
8. Common Issues & Debug Tips	9
GTKWave Tips	10
Debug Technique: Binary Search in Waveform.....	10
9. Summary	10
Interview Question Topics from Waveform Analysis	11

Waveform Analysis Using Testbenches

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

Waveform Analysis Using Testbenches — Debugging RTL and Verifying Protocol Compliance

2. Objective

By the end of this module, learners will be able to:

- Use GTKWave to navigate large simulation waveforms (VCD/FST format)
- Identify AXI-Lite valid/ready handshake patterns in waveforms
- Verify UART 8N1 frame format by measuring bit-period timing
- Correlate simulation log output with waveform events
- Use structured waveform analysis to debug protocol violations and timing issues
- Organize GTKWave signal groups for efficient multi-layer SoC inspection

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

A waveform viewer is like a slow-motion camera for your digital circuit. Every wire in your design is recorded over time, so you can go back and ask:

- “At exactly this moment, what was the CPU doing?”
- “Did the UART transmit ‘H’ before or after the AXI handshake?”
- “Why did the bridge get stuck in state 3?”

GTKWave reads .vcd (Value Change Dump) or .fst (Fast Signal Trace) files produced by Verilator or Icarus Verilog simulations. These files record every signal transition in the simulation.

3.2 Intermediate — Engineering View

VCD vs FST: | Format | File size | Load speed | Tool | |----|-----|-----|----| | .vcd | Large (147 MB for tb_top.vcd) | Slow | Any viewer | | .fst | Compressed (~5 MB) | Fast | GTKWave |

Always convert before viewing:

```
vcd2fst tb_top.vcd tb_top.fst    # ~10-20x smaller  
gtkwave tb_top.fst &
```

Available waveform files in this project:

File	Source	Contains
tb_top.vcd / tb_top.fst	make soc	Full SoC simulation (primary)
tb_top_debug.vcd / tb_top_debug.fst	make debug_soc	Debug TB with extra logging
tb_axi_lite.vcd	make axi_uart	AXI-UART module test
tb_axi_lite_stand alone.vcd	make axi_sa	AXI handshake demo
tb_riscv_axi_lite .vcd	make riscv_axi	CPU-AXI integration demo
tb_uart.vcd	make uart	UART loopback test
tb_picorv32.vcd	make pico	PicoRV32 standalone
tb_mem_soc.vcd	make soc_pico_uart_mem	Direct-bus MMI SoC

3.3 Advanced — Signal/Architecture Level

Testbench structure in tb/tb_top.v:

Three concurrent processes run simultaneously in the testbench: 1. **Clock generator:** always #10 clk = ~clk (50 MHz) 2. **UART TX monitor** (initial begin : uart_monitor): Decodes serial bitstream into bytes using bit-time sampling (BAUD_HALF, BAUD_FULL delays) 3. **Byte logger** (initial begin : byte_logger): Prints [TB] RX byte: X for every FIFO entry — runs independently to avoid race conditions 4. **Test sequencer** (initial begin : test_seq): Waits for 10 greetings, sends PING, verifies echo

UART Monitor timing constants:

```
localparam CLK_PERIOD      = 20;           // 20 ns → 50 MHz
localparam UART_BIT_CYCLES = 434;         // cycles per bit @115200, 50MHz
localparam BAUD_HALF      = 1_000_000_000 / 115200 / 2; // 4340 ns
localparam BAUD_FULL      = 1_000_000_000 / 115200;    // 8680 ns
```

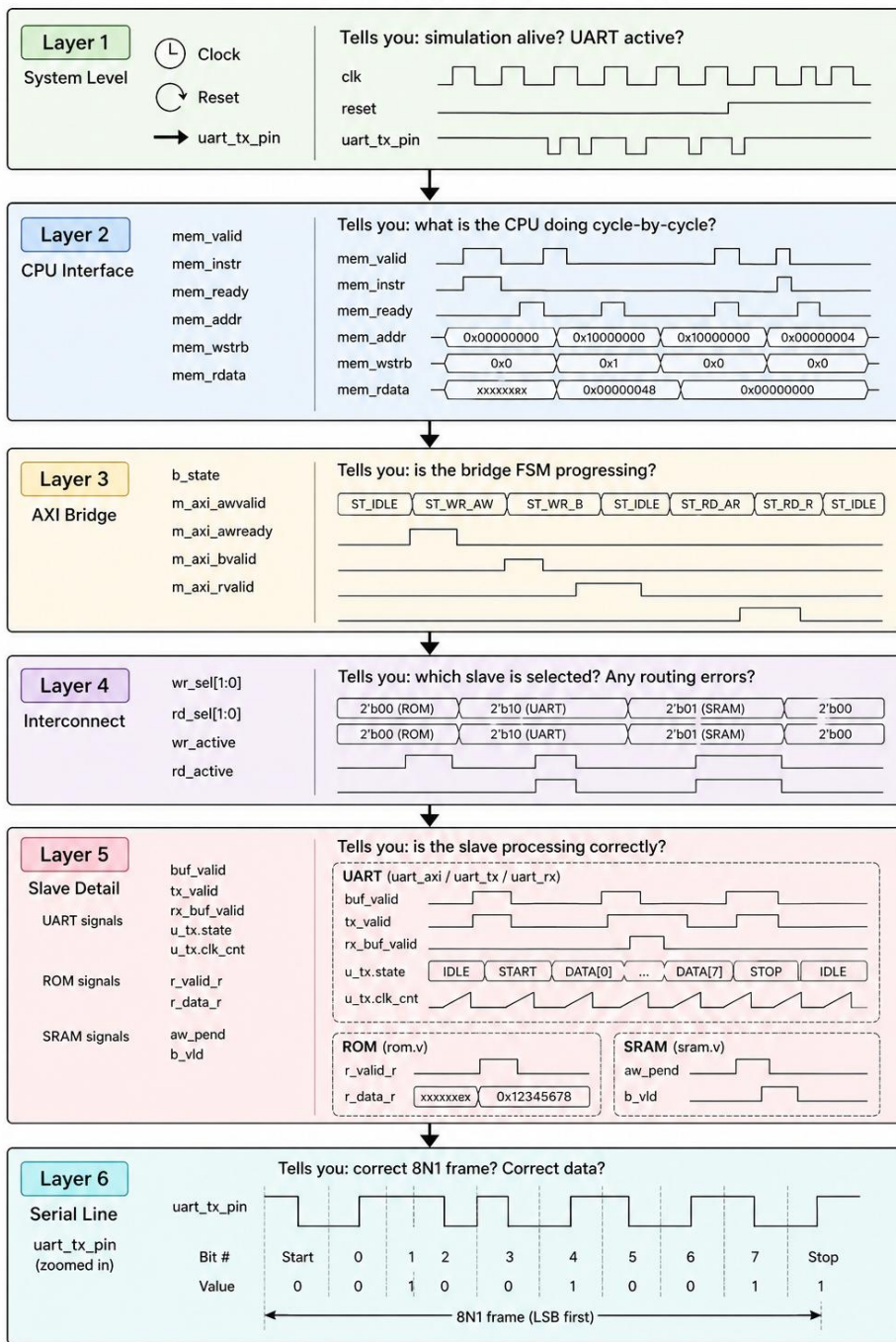
UART monitor algorithm:

```
forever begin
    @(negedge uart_tx_pin);           // 1. Wait for start-bit falling edge
    #(BAUD_HALF);                     // 2. Jump to mid-start-bit
    if (uart_tx_pin == 1'b0) begin    // 3. Confirm genuine start bit (not
glitch)
        for (bit_i = 0; bit_i < 8; bit_i++) begin
            #(BAUD_FULL);             // 4. Jump to mid-bit of each data bit
            mon_byte[bit_i] = uart_tx_pin;
        end
        #(BAUD_FULL);                // 5. Skip stop bit
        rx_fifo[rx_wr_ptr & 63] = mon_byte;
    end
end
```

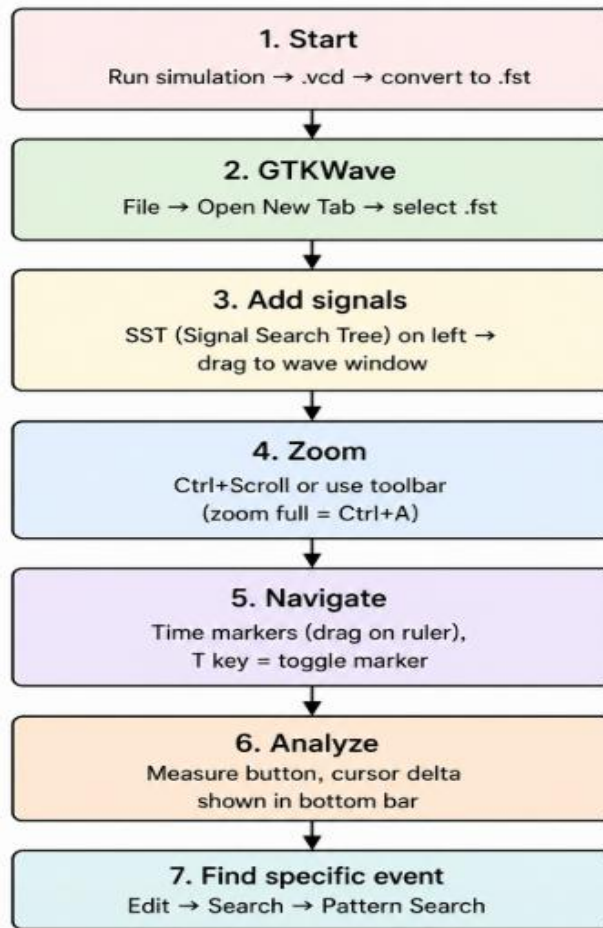
```
rx_wr_ptr++;  
end  
end
```

4. Architecture / Concept

GTKWave Analysis Layers



Waveform Navigation Workflow



5. Implementation

Testbench Files for This Module

File	Key Features
tb/tb_top.v	UART bit decoder, 64-byte FIFO, byte logger, PING test
tb/tb_debug.v	Additional debug signal display; structured test log format
tb/tb_axi_lite_standalone.v	Clean AXI handshake demo with \$display labels
tb/tb_riscv_axi_lite.v	Step-by-step AXI trace with human-readable messages
uart.gtkw	Pre-saved GTKWave configuration for UART signals

Pre-saved GTKWave Layout

```
gtkwave tb_uart.vcd -a uart.gtkw &
```

This opens GTKWave with pre-configured signal groups for UART analysis.

6. Lab / Hands-on Section

Lab A: Full SoC Waveform Analysis

Step 1: Generate the waveform

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc  
make soc
```

Step 2: Convert and open

```
vcd2fst tb_top.vcd tb_top.fst  
gtkwave tb_top.fst &
```

Or simply:

```
make wave # Does vcd2fst + gtwave automatically
```

Step 3: Add signal groups

In GTKWave's SST (left panel), expand the hierarchy:

```
tb_top  
├── u_dut (top module)  
│   ├── (signals in top.v: b_state, m_axi_*)  
│   ├── u_cpu (picorv32)  
│   │   ├── mem_valid, mem_addr, mem_rdata, ...  
│   ├── u_interconnect  
│   │   ├── wr_sel, rd_sel, wr_active, ...  
│   ├── u_rom  
│   │   ├── r_valid_r, r_data_r  
│   ├── u_sram  
│   │   ├── aw_pend, b_vld  
│   └── u_uart  
│       ├── buf_valid, aw_got, w_got  
│       ├── u_tx (uart_tx)  
│       │   ├── state, clk_cnt, bit_cnt, shift_reg, tx  
│       └── u_rx (uart_rx)  
│           ├── state, rx_buf_valid, data_out
```

Step 4: Zoom to first UART write

Use Edit → Search → Timestamp or Ctrl+G to jump to time $\approx 5 \mu\text{s}$ (first UART TX byte).

Step 5: Decode the first byte manually

In GTKWave, zoom into uart_tx_pin signal: - Find the falling edge (start bit) - Count 8 data bits, each lasting $\approx 434 \times 20 \text{ ns} = 8680 \text{ ns}$ - Sample mid-bit: start + 4340 ns, + 8680 ns $\times n$ - Build byte from LSB to MSB - Verify it equals 0x48 ('H')

Lab B: AXI Handshake Analysis

`make axi_sa`
`gtkwave tb_axi_lite_standalone.vcd &`

Exercises:

1. **Measure AW handshake latency:**
 - Place Marker 1 at the cycle where awvalid rises
 - Place Marker 2 at the cycle where awready rises
 - Delta = handshake latency (should be 1 cycle in this testbench)
2. **Find Write Response:**
 - After W handshake: how many cycles until bvalid=1?
 - Expected: 1 cycle (slave asserts bvalid immediately)
3. **Verify wstrb:**
 - What is wstrb during the write? (Should be 0xF = 4'b1111)
4. **Confirm data integrity:**
 - What is wdata at the W handshake? (0xA5A5A5A5)
 - What is rdata at the R handshake? (Should match)

Lab C: Protocol Debug Scenario

`make riscv_axi`
`gtkwave tb_riscv_axi_lite.vcd &`

Compare the terminal output with the waveform: - Find "Step 1: Send Address" → locate `m_awvalid=1` in waveform - Find "AWREADY = 1" → find the exact cycle `s_awready` rises - Find "BVALID = 1" → locate `s_bvalid` transition

This exercise teaches correlation between \$display output (timing logs) and actual waveform transitions.

7. Waveform / Verification Insight

Checklist: What to Verify in tb_top.fst

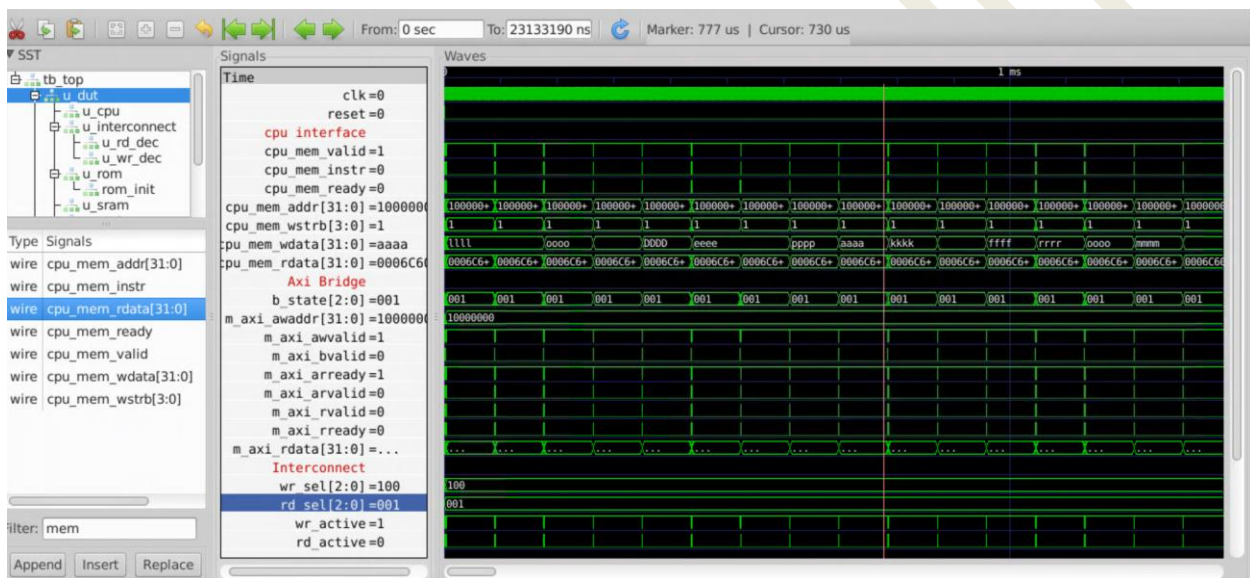
System initialization: - [] `reset=1` for first 10 cycles - [] After `reset=0`: `b_state=0` (ST_IDLE), `m_axi_awvalid=0` - [] First `mem_addr=0x0000_0000` (instruction fetch)

Instruction fetch verification: - [] `mem_instr=1` on all instruction fetches - [] `rd_sel=3'b001` (ROM selected) for first 50+ accesses - [] `mem_rdata` matches entries in `rtl/rom.hex`

AXI write to UART: - [] m_axi_awaddr=0x1000_0000 when CPU writes to UART_TX - [] wr_sel=3'b100 (UART selected) at that moment - [] s2_awvalid=1 (only UART's AW channel asserted, not S0/S1) - [] u_uart.aw_got=1 then w_got=1 → buf_valid=1 → u_tx.valid=1

UART TX frame: - [] uart_tx_pin falls to 0 (start bit) - [] 8 data bits, each lasting 434 cycles (8680 ns) - [] Returns to 1 (stop bit), then stays 1 (idle) - [] Total frame = 4340 cycles = 86.8 μs

PING echo test: - [] uart_rx_pin driven by testbench (falls to 0 for 'P', 'I', 'N', 'G') - [] u_uart.u_rx.data_valid pulses for each received byte - [] CPU reads status 0x1000_0008 until rx_buf_valid=1 - [] CPU reads 0x1000_0004 → gets received byte - [] CPU writes received byte back to 0x1000_0000 → echo



Timing Reference Table

Event	Simulated Time
Reset released	~200 ns
First instruction fetch	~220 ns
First UART byte 'H' starts	~3–5 μs
First greeting 'H' fully serialized	~90 μs
All 10 greetings complete	~23 ms
PING bytes sent by TB	~23 ms
TEST PASSED	~23.4 ms

8. Common Issues & Debug Tips

Issue in Waveform	Meaning	Fix
b_state stuck at non-zero	AXI bridge deadlock	Check which slave's handshake is missing valid or ready
wr_sel=3'b000	Address decoder default triggered	CPU accessed an unmapped address; check mem_addr
uart_tx_pin always 1	No data transmitted	Check buf_valid and aw_got/w_got in u_uart
Short glitch on uart_tx_pin	Not a start bit (< BAUD_HALF)	Monitor validates: it re-checks mid-bit, ignores glitches
m_axi_rdata always 0x13	ROM returning NOP	ROM not loaded correctly — check [ROM] loaded from... log
No bvalid pulse	Slave never responds	Expand slave signals — check if slave received awvalid+wvalid
mem_instr=0 for many cycles	CPU in memory access	Normal: load/store operations; check address
Simulation ends unexpectedly	Timeout watchdog fired	Check [TB] ERROR: Simulation TIMEOUT in log

GTKWave Tips

Ctrl+A → Zoom to fit all signals
 Ctrl+Scroll → Zoom in/out at cursor
 T → Place/cycle through time markers
 Ctrl+G → Go to specific time
 Shift+Click → Extend selection
 Color groups → Right-click signal → Highlight → Color (improves readability)
 Signal search → Edit → Search → Signal Search Tree

Debug Technique: Binary Search in Waveform

Symptom: "TEST FAILED" output

1. Jump to conclusion time → check echo[0..3] vs expected PING
2. If echo = 0x00: CPU never echoed → trace uart_putc calls
3. If echo[0] wrong: trace UART RX path, check rx_buf_data
4. If CPU never called uart_getc: trace uart_getc status poll
5. Trace backwards to first wrong event → root cause

9. Summary

Skill	Tool / Technique
VCD compression	vcd2fst tb_top.vcd tb_top.fst → 30× size reduction
Signal navigation	SST hierarchy in GTKWave → expand instances
AXI verification	Look for simultaneous VALID+READY in both channels
UART decoding	Measure 434-cycle bit periods; build byte LSB-first
Bridge debug	Monitor b_state[2:0] for FSM stuck conditions
Address routing	Monitor wr_sel/rd_sel one-hot signals in interconnect
Latency measurement	GTKWave time markers (T key) for cycle-accurate measurement
Waveform files	8 different VCD files covering each test scenario

Interview Question Topics from Waveform Analysis

1. **“How do you verify an AXI handshake?”** → Both VALID and READY must be 1 at the same rising edge
2. **“How do you debug a UART framing error?”** → Measure bit period in GTKWave; verify mid-bit sampling alignment
3. **“How do you find a stalled AXI transaction?”** → VALID=1 but READY never responds → check slave state machine
4. **“What is the latency from CPU write to UART bit?”** → Bridge latency (3 cycles) + UART start-bit (434 cycles) = ~437 cycles = 8.74 μs at 50 MHz
5. **“How do you check the address decoder is correct?”** → Monitor wr_sel/rd_sel one-hot and verify it changes correctly with mem_addr

Previous: [Sub-Module 8 — Full SoC Integration](#)

Back to: [Master Index](#)

This module serves as the verification reference for the entire course. Keep these waveform files (tb_top.fst, tb_top_debug.fst, tb_axi_lite_standalone.vcd, tb_riscv_axi_lite.vcd) as your debugging toolkit.